XRPL Validator — Daily Engineering Report · 2026-08-21

Period: 2026-08-21 00:00 to 24:00 US Eastern (2026-08-21 04:00 to 2026-08-22 04:00 UTC). Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: xrpl-validator merge records (they begin 2026-07-04). Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Step 1 receipt records (they begin 2026-08-29); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); amendment data; data.xrpl.org agreement scores (scored days begin 2026-04-04; some days have none).

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** no receipt records for this day (they begin 2026-08-29).
- **Changes:** 0 deployments, 1 commit merged into main.

Service levels

No signing records for this day (they begin 2026-09-19).

Native engine conformance

No receipt records for this day (Step 1 records begin 2026-08-29, Steps 2–5 records 2026-10-09).

Live counters are kept only for the current run; none exist for a past day.

Receipt register

No receipt records for this day (Step 1 records begin 2026-08-29).

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

Merged	Repo	Commit	Subject
02:02 UTC	xrpl-sdk	`e8a2acb`	fix(core): full-state codec parity — STAmount getText spelling, MPT amounts, BaseTen u64s, 62-bit XRP range

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **feat(xrpl-ledger): PreviousTxnID/PreviousTxnLgrSeq stamping + post-state threading census** (`9876271`, xrpl-validator) Byte-for-byte phase 1: the engine now THREADS. New ledger::threading — stamp_threading marks every materially-changed write with the touching tx's hash and the ledger seq, under rippled's ApplyStateTable rule: a content-equal write-back (modulo pointer spelling and existing threading) is dropped from the meta and never threaded, so it is never stamped. 28 of 30 entry types carry the fields (fixPreviousTxnID added …
- **feat(xrpl-ledger): threadOwners — created/deleted nodes thread their owner roots** (`48367bc`, xrpl-validator) Both census finds, one rule (ApplyStateTable.cpp:640-668): every CREATED or DELETED node threads the transaction to its owner accounts' AccountRoots — both limit issuers for a RippleState, else sfAccount and sfDestination when present, nothing for an AccountRoot. Those roots become the meta's pure-threading ModifiedNodes: the FinalFields==pre refreshes the expected-side filter has always dropped.
- **feat(xrpl-ledger): MPT issuance lifecycle — Create/Destroy/Set/Authorize, stubs retired** (`64ebb07`, xrpl-validator) Byte-for-byte phase 2, first tranche: the four MPT lifecycle transactors leave STUB_TYPES and are judged like everything else.
- **feat(xrpl-ledger): DID, PermissionedDomain, AMMClawback — five more stubs retired** (`75b2517`, xrpl-validator) Zero mainnet specimens exist for any of these in every fixture window, so these are BLIND SOURCE PORTS — flagged as such in-code; the scout becomes the verifier the day one lands. A faithful port can silently match; a fee-only stub is GUARANTEED to diverge.
- **feat(xrpl-ledger): XChain — all eight XLS-38 transactors; STUB_TYPES is EMPTY** (`6036cee`, xrpl-validator) The last eight stubs. ZERO mainnet usage exists (no Bridge object on mainnet, no XChain tx in any fixture window), so the only reachable branch today is 'no bridge' (tecNO_ENTRY) — everything deeper

executes only after a XChainCreateBridge lands, which the scout would surface with a fixture. Blind source ports from XChainBridge.cpp (2390 lines, all eight transactors in one file): - keylets: bridge = …

- **feat(xrpl-node): BYTE CENSUS — canonical-codec serialization vs ledger_entry binary=true** (`a0b42aa` , xrpl-validator) Byte-for-byte phase 3, the instrument: DX_BYTECHECK=1 serializes every surviving touched node through the SDK's canonical STObject codec (xrpl_core::codec::encode_transaction_json — its definitions table covers ledger fields, LedgerEntryType string resolution, Vector256, inner arrays/objects) and byte-compares against the real post-state blob from ledger_entry binary=true. Reports first-diff offset with hex context, …
- **feat(xrpl-ledger): byte backlog to ZERO — 98→0, ten fixtures byte-exact** (`1733d67` , xrpl-validator) Three grind rounds against the byte census, every fix a canonical-shape rule the meta-level gates could never see: ROUND 1 (98→40): directory zero-link pollution — relinks wrote explicit 'IndexNext: 0' onto hydrated pages and every fresh page carried both zero links; fresh pages now omit them (set_link). Fresh BOOK pages gain their canonical fields (ExchangeRate = the root key's low 8 bytes + TakerPays/Gets …
- **feat(xrpl-ledger): pseudo-transactions + flag-ledger NegativeUNL rotation — phase 4 opens** (`1083ba5` , xrpl-validator) UNLModify / SetFee / EnableAmendment (Change.cpp), the consensus-injected transactions with no Account, no Fee, no Sequence. dispatch::is_pseudo routes them past preclaim/apply_common entirely; build_txfields accepts the zero account. New keylets: amendments (dedup'd — one already existed), negativeUNL. The NegativeUNL/Amendments singletons hydrate on every fixture alongside FeeSettings.
- **fix(xrpl-ledger): self-cross deletions roll back with a failed strand — dead reaps stay banked** (`9475a92` , xrpl-validator) Scan 4's single find, l106453801: a mirrored mis-attribution. 2D468DC2's REJECTED bridge candidate had walked leg A and self-cross-deleted the taker's own resting offer A20C602E (plus its book page); our rollback re-applied every reap from the failed candidate, so the deletion landed in 2D468DC2's meta (8v6 extra) — while mainnet still holds the offer until EA6D14A1 places on that very book ten txs later and …
- **feat(xrpl-node): state_verify — snapshot-dir account_hash verification** (`05de843` , xrpl-validator) Phase 4 substrate check: reads snapshot.py's harvested batches ({index, data} hex pairs), builds the State SHAMap with the canonical leaf formula (SHA512Half(MLN\0 || data || key) — verify_rocks' and the live validator's), and compares the computed root against the pinned ledger's account_hash. Proves snapshot + SHAMap + hashing end-to-end before anything replays on top.
- **fix(core): full-state codec parity — STAmount getText spelling, MPT amounts, BaseTen u64s, 62-bit XRP range** (`e8a2acb` , xrpl-sdk) Fixes surfaced by replaying mainnet ledger #106455035 over the full 19,376,944-object state snapshot (decode → canon → encode must reproduce every blob byte-for-byte; census now 0 mismatches, 0 decode errors): - IouAmount::to_decimal mirrors STAmount::getText exactly: exponents outside [-25, -5] (and != 0) print as 'e' instead of a plain expansion the parser could not re-read (32,442 objects). - …
- **feat(node): state_replay — full-ledger native replay vs mainnet account_hash** (`83858f9` , xrpl-validator) Phase 4 capstone. Loads the entire 19,376,944-object verified snapshot, decodes every blob through the canonical codec (the load doubles as a full-state codec census: decode → canon → encode must reproduce every blob — 0 mismatches after the xrpl-core amount fixes), hexifies to the engine's account convention, applies the next ledger's tx set through the native engine with NO truth-overlay and NO hydration, stamps …
- **fix(xrpl-ledger): six mainnet-parity fixes from full-ledger replay rounds 2-3** (`e84d0b3` , xrpl-validator) Replaying #106455035-37 against account_hash (state_replay) surfaced six engine gaps the fixture corpus never reached; with all six the streak is three consecutive ledgers byte-for-byte (518 txs): - payment: a LEADING issuer-only path element (type 32) seeds from the spend currency — toStrand's normalization walks src → SendMax issuer first and the pairwise emission builds the same-currency cross-issuer book. …

- **fix(xrpl-ledger): tfSell residual rests the subtracted in-side, repriced out** (`26b2e2d`, xrpl-validator) OfferCreate.cpp:491-521: after crossing, a tfSell offer's remainder keeps afterCross.in = TakerGets minus the crossed NET (a 16-digit STAmount subtraction), and REPRICES the out side — divRoundStrict(afterCross.in, rate, out.asset, roundUp=false), whole-drop floor on an XRP leg. Our placement derived the GETS side from rem_pays for both modes (right for buys — E7399DA3 — backwards for sells).

Independence program

No independence record for UTC day 2026-08-21; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

No amendment records for this day.

Open items

None.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Oracle-Triggered Orders via Sorted Directories** (dex) Add a tfTrigger flag to OfferCreate plus two SFields, sfTriggerPrice and sfOracleID (a keylet reference into an existing PriceOracle), creating a Stop
- **Sponsorship Transfer for Client Offboarding** (custody) Add a SponsorTransfer transaction (ttSPONSOR_TRANSFER) carrying sfObjectID and sfNewSponsor. Preclaim requires the incoming sponsor to hold a standing
- **Oracle-Ordered Liquidation Fencing for Vaults** (lending) Add a monotonic sfUpdateCounter SField to PriceOracle, incremented exactly once per ledger in which it is updated, with the increment order fixed by f

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
08-21 11:42	—	xrpl-validator	`9876271`	feat(xrpl-ledger): PreviousTxnID/PreviousTxnLgrSeq stamping + post-state threading census
08-21 12:20	—	xrpl-validator	`48367bc`	feat(xrpl-ledger): threadOwners — created/deleted nodes thread their owner roots
08-21 12:45	—	xrpl-validator	`64ebb07`	feat(xrpl-ledger): MPT issuance lifecycle — Create/Destroy/Set/Authorize, stubs retired
08-21 16:05	—	xrpl-validator	`75b2517`	feat(xrpl-ledger): DID, PermissionedDomain, AMMClawback — five more stubs retired
08-21 16:16	—	xrpl-validator	`6036cee`	feat(xrpl-ledger): XChain — all eight XLS-38 transactors; STUB_TYPES is EMPTY
08-21 16:46	—	xrpl-validator	`a0b42aa`	feat(xrpl-node): BYTE CENSUS — canonical-codec serialization vs ledger_entry binary=true
08-21 21:28	—	xrpl-validator	`1733d67`	feat(xrpl-ledger): byte backlog to ZERO — 98→0, ten fixtures byte-exact
08-21 22:41	—	xrpl-validator	`1083ba5`	feat(xrpl-ledger): pseudo-transactions + flag-ledger NegativeUNL rotation — phase 4 opens
08-21 22:51	—	xrpl-validator	`9475a92`	fix(xrpl-ledger): self-cross deletions roll back with a failed strand — dead reaps stay banked
08-21 23:26	—	xrpl-validator	`05de843`	feat(xrpl-node): state_verify — snapshot-dir account_hash verification
08-22 02:02	08-22 02:02	xrpl-sdk	`e8a2acb`	fix(core): full-state codec parity — STAmount getText spelling, MPT amounts, BaseTen u64s, 62-bit XRP range
08-22 02:02	—	xrpl-validator	`83858f9`	feat(node): state_replay — full-ledger native replay vs mainnet account_hash
08-22 02:56	—	xrpl-validator	`e84d0b3`	fix(xrpl-ledger): six mainnet-parity fixes from full-ledger replay rounds 2-3
08-22 03:21	—	xrpl-validator	`26b2e2d`	fix(xrpl-ledger): tfSell residual rests the subtracted in-side, repriced out

Times in UTC. A commit appears on the day it was committed to main or merged into it.